File Upload Security Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ File Upload — Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use a local lab or authorized staging environment. Do not upload executable payloads to production systems.*

■ Executive Summary

File-upload flaws occur when an application trusts client-controlled filenames, extensions, MIME types, or file content. The supplied PDF covers MIME changes, extension variations, filename edge cases, size validation, and risks associated with different formats.

■ Core Testing Methodology

1. Map upload features, permitted formats, size limits, and storage behavior.
2. Upload a harmless file of an allowed type using a test account.
3. Verify server-side content validation rather than trusting the client `Content-Type`.
4. Check filename normalization and ensure paths cannot be changed or unrelated files overwritten.
5. Verify size, image-dimension, and processing-time limits.
6. Confirm uploads cannot execute as server-side code and private files require authorization.
7. Use harmless malformed-file and MIME-mismatch fixtures in staging.
8. Record redacted evidence and retest the fix.

■ High-Value Bug Bounty Tips & Edge Cases

- The PDF shows changing multipart content types; these values are client-controlled.
- It lists double extensions, uncommon extensions, mixed capitalization, null-byte and path-like filename ideas. Treat these as legacy test categories, not guaranteed bypasses.
- Content-length controls should be enforced at proxy and application layers.
- Images, SVG, HTML, XML, CSV, and document formats have different processing risks.
- Neither an extension allowlist nor a blocklist replaces content validation and safe storage.
- The source includes executable upload examples; they are intentionally omitted here.

■■ Educational Checklists & Safe Examples

```
POST /uploads HTTP/1.1
Host: example.com
Content-Type: multipart/form-data; boundary=[BOUNDARY]

--[BOUNDARY]
Content-Disposition: form-data; name="file"; filename="sample.txt"
Content-Type: text/plain

Harmless test document.
--[BOUNDARY]--
```

- [ ] Actual file content is validated.
- [ ] Upload size and processing limits are enforced.
- [ ] Server-generated filenames are used.
- [ ] Upload directories do not execute scripts.

- [ ] Private uploads require authorization.
- [ ] Active content is sanitized or served from an isolated origin.

■■ Mitigation & Remediation

Use a strict allowlist, validate signatures and parse content with maintained libraries, generate safe filenames, store uploads outside executable web roots, impose size and processing limits, re-encode images when appropriate, isolate active content, and enforce authorization on downloads.

■ Source Notes

Based on the supplied four-page *FILE UPLOAD.pdf*. Pages 1–3 cover content-type and extension variations, size checks, filename cases, format risks, and allow/blocklist concepts. Dangerous executable payloads are not reproduced.

Educational use only. Test uploads in a controlled environment.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.